

SUMMER RESEARCH INTERNSHIP PROGRAMME

2026

FINAL RESEARCH REPORT — SUBMISSION COVER PAGE

PROJECT / RESEARCH TITLE

[NashDQNSleep+: A Non-Linear AoI Model for Better Sleep Scheduling]

PROGRAMME DETAILS

Domain / Track:	<i>IoT</i>	Duration:	<i>15th May – 15th August</i>
Department:	<i>Industrial IoT</i>	Batch / Year:	<i>3rd Year (5th Semester)</i>

SUBMITTED BY

Student 1	<i>Swastik Basu</i>
Student 2	<i>[Full Name]</i>

UNDER THE MENTORSHIP OF

Mentor Name	<i>Prof. Debashis De</i>
Affiliation	<i>[Department / Organisation]</i>

DECLARATION

We hereby declare that this report is a record of genuine work carried out by us under the guidance of the above-mentioned mentor, as part of the IEEE SMC Student Branch Chapter, KGEC — Research Internship Programme 2026, and has not been submitted elsewhere for any other purpose.

Mentor's Signature

Programme Coordinator

IEEE SMC Student Branch Chapter, KGEC | Research Internship Programme 2026 | Date of Submission: *[20 / 08 / 2026]*

NashDQNSleep+: A Non-Linear Age of Information Model for Better Sleep Scheduling in Industrial IoT

Swastik Basu

Department of Computer Science and Engineering
Ramakrishna Mission Vivekananda Centenary College,
Rahara, West Bengal

Summer Research Internship — IEEE SMC KGEC Student Branch

Abstract— This report examines a recently published algorithm, NashDQNSleep, which addresses a scheduling problem in Industrial Internet of Things (IIoT) networks. Sensor nodes must determine when to enter sleep mode in order to conserve battery energy; however, prolonged sleep intervals result in the transmission of outdated data. NashDQNSleep addresses this trade-off through game theory and deep reinforcement learning. A detailed examination of the underlying formulation reveals several limitations, the most significant of which concerns the manner in which data staleness is quantified. The original work employs Age of Information (AoI) as its staleness metric but models AoI growth as a fixed increment at every time step. This linear formulation does not correspond to the manner in which stale data affects system performance in practice, where the associated risk increases more rapidly than a constant rate would suggest. Accordingly, this report proposes an extension, designated NashDQNSleep+, in which AoI growth is modelled non-linearly using an exponential function. A Python-based simulation was developed to evaluate this extension and to compare its performance against the original formulation.

Index Terms—Age of Information, IIoT, Sleep Scheduling, Deep Q-Network, Nash Equilibrium, Non-Linear AoI.

I. INTRODUCTION

Industrial environments increasingly rely on distributed sensor nodes to collect data and relay it to a central monitoring system for real-time analysis. These sensors are battery-powered, which imposes a fundamental trade-off: continuous operation depletes battery reserves rapidly, whereas excessive sleep duration causes transmitted data to become outdated and therefore less useful for monitoring purposes.

The staleness of transmitted data is commonly quantified using Age of Information (AoI). If a sensor last transmitted an update ten time steps ago, its AoI is equal to ten; lower AoI values indicate fresher data. Banerjee, Goswami, and De [1] proposed an algorithm, NashDQNSleep, that employs game theory and deep reinforcement learning to enable each sensor node to determine autonomously when to sleep. The reported results indicate substantial improvements, including a thirty-five percent reduction in energy consumption and a forty percent reduction in AoI relative to prior methods.

A detailed review of the mathematical formulation in [1], however, indicates that AoI is modeled as increasing by exactly one unit at every time step in which no data is transmitted. This constitutes a linear model. In practical deployments, the consequences of stale data tend to compound more rapidly than a linear model would predict. This report examines this limitation and proposes a corresponding modification.

This report is organized as follows. First, it identifies the research gaps present in the original formulation. Second, it presents the proposed engineering solution, which replaces the linear AoI growth equation with an exponential one. Third, it presents simulation results comparing the performance of the original and proposed formulations.

II. BACKGROUND

A. What NashDQNSleep Does

The idea in [1] is that each sensor node in the network is a player in a game. Every node wants to maximize its own reward, which is a mix of having good battery life and low AoI. The paper uses Deep Q-Networks (DQN) to help each node learn when to sleep and when to wake up. The nodes also share information with nearby nodes in a peer-to-peer way, so they do not need a central controller.

The state of a node at time step k is defined in [1, Eq. 16] as:

$$S_n(k) = \{ B_n(k), s_n(k-1), \Delta_n(k), \Delta T_n(k) \}$$

where $B_n(k)$ is battery level, $s_n(k-1)$ is whether it was awake last step, $\Delta_n(k)$ is AoI, and $\Delta T_n(k)$ is temperature increase. The reward function from [1, Eq. 17] is:

$$R_n(k) = w_1 \cdot B_n(k) - w_2 \cdot \Delta_n(k) - w_3 \cdot \max(0, \Delta_n(k) - \Delta_{th}) - w_4 \cdot (N_n(k) - 1)^2 \cdot I(N_n(k) > 1) + w_5 \gamma_n(k) \cdot s_n(k)$$

where the weights w_1 through w_5 balance energy, AoI, penalty for very high AoI, collision avoidance, and transmission success.

B. The AoI Update Rule in the Paper

This is the key equation I am focused on. The paper says in [1, Eq. 15, p. 4]:

$$\begin{aligned} \Delta_n(k+1) &= 1, & \text{if } s_n(k)=1 \text{ and } \gamma_n(k)=1 \\ \Delta_n(k+1) &= \Delta_n(k) + 1, & \text{otherwise} \end{aligned}$$

This means if a transmission fails or the node is asleep, AoI just goes up by 1. That is it. The same amount every step. It does not matter if the AoI is already very high.

III. RESEARCH GAPS FOUND IN NASHDQNSLEEP

After reading the full paper and comparing it with other papers, I found seven gaps. I will explain each one.

A. Gap 1: AoI Grows in a Linear Way (Main Gap)

As shown in Eq. 15 of [1], AoI increases by 1 every time step. This is a linear model. But think about it this way: if your phone's GPS map was last updated 2 minutes ago, that is fine. But if it was last updated 2 hours ago and you are driving on a highway, that is a much bigger problem than simply 60 times worse. The danger grows faster than the time difference.

In IIoT, the same idea applies. A sensor reading that is 10 steps old could be much more dangerous than 10 times a 1-step-old reading, especially in safety-critical applications like gas leak monitoring or predictive maintenance. The paper does not capture this. Raghuwanshi et al. [2] made the same observation and proposed a non-linear AoI model specifically for IIoT scenarios.

B. Gap 2: Battery Only Goes Down

In [1, Eq. 13], the battery just decreases at every step. There is no way for it to go up. But in the real world, many IIoT sensors use solar panels or RF energy harvesting to recharge. So the model in the paper cannot be applied to those kinds of sensors at all. Hatami et al. [5] already solved AoI scheduling for energy-harvesting sensors, so this is a known gap.

C. Gap 3: No Proof That the Algorithm Converges

The paper claims Nash equilibrium is reached through the DQN training [1, Eq. 50–53, pp. 9–10]. But it only shows this through simulation results. There is no mathematical proof that the algorithm will actually converge in all cases. When multiple agents are learning at the same time, each agent's environment keeps changing because the other agents are also updating. This breaks the standard assumption that DQN needs to

work correctly. Li and Ding [9] recently published a paper that does give convergence guarantees for multi-agent games, which shows it is possible but the original paper did not do it.

D. Gap 4: Neighbour States Are Guessed, Not Known

The Nash Q-function in [1, Eq. 51] depends on knowing what other nodes are doing. But the paper itself admits on page 10 that these actions are not directly observable and have to be estimated using messages passed between nodes. If those messages are delayed or lost, the estimates will be wrong and the reward calculation will be off. There is no POMDP (Partially Observable Markov Decision Process) or recurrent model to handle this uncertainty properly.

E. Gap 5: All Nodes Are Treated the Same

The state vector and reward weights in [1, Eq. 16–17] are identical for every single node. But in a real factory, you might have tiny temperature sensors, larger vibration sensors, and gateway nodes, all with very different battery sizes and data priorities. Treating them all the same means the algorithm cannot adapt to that kind of heterogeneity. Wang et al. [12] showed that using different model structures for different node types gives much better results.

F. Gap 6: Only Tested in Simulation

All results in the paper come from a Python Gym simulation [1, Section 7]. The hardware implementation in Section 7.6 is just a block diagram — it was never actually built or tested. Numbers like “0.08ms per forward pass” are calculated, not measured. Li et al. [14] showed that real-world network management using DRL behaves quite differently from simulation because of things like packet loss timing and hardware interrupts.

G. Gap 7: Multi-Hop Relaying Is Ignored in the Game

This one is actually an internal contradiction in the paper itself. In [1, Eq. 5, p. 4], the energy model includes relay forwarding — meaning a node spends energy forwarding other nodes' packets. But the reward function [1, Eq. 17] and the Nash equilibrium setup [1, Eq. 50] never penalize a node for sleeping when it is acting as a relay for others. So a relay node could decide to sleep, and the game would not care, even though this could cut off other nodes from reaching the sink. Zakeri et al. [16] specifically studied this in two-hop relay systems and found it significantly changes the optimal policy.

IV. PROPOSED SOLUTION: NASHDQNSLEEP+

My proposed solution is called NashDQNSleep+. The name stands for Nash-based Deep Q-Network with

Non-linear AoI and Relay-Aware Scheduling. For this paper, I am focusing on fixing Gap 1, which is the most fundamental one because it affects the core reward and learning signal of the whole algorithm.

A. The New AoI Update Equation

I replace [1, Eq. 15] with what I call Eq. 15'. Instead of adding a constant 1 every step, I add an amount that grows as AoI grows. The new equation is:

$$\Delta_n(k+1) = 1, \quad \text{if } s_n(k)=1 \text{ and } \gamma_n(k)=1$$

$$\Delta_n(k+1) = \Delta_n(k) + e^{\lambda \cdot (\Delta_n(k)-1)}, \quad \text{otherwise}$$

Here, λ is a parameter I can tune. When $\lambda = 0$, the equation becomes exactly the same as the original. So the old NashDQNSleep is a special case of NashDQNSleep+ when $\lambda = 0$. This makes it backward compatible. When $\lambda > 0$, AoI grows faster and faster as it gets larger, which reflects what actually happens in practice.

B. The New Reward Penalty Function

Because AoI is now non-linear, I also change how the reward penalizes it. In the original paper, the penalty is just $-w_2 \cdot \Delta_n(k)$, which is proportional to AoI. In NashDQNSleep+, I use a convex cost function called $\varphi(\Delta)$, defined as:

$$\varphi(\Delta) = (e^{(\lambda \cdot \Delta)} - 1) / \lambda \quad \text{for } \lambda > 0$$

$$\varphi(\Delta) = \Delta \quad \text{for } \lambda = 0$$

The modified reward function (Eq. 17') becomes:

$$R_{n+}(k) = w_1 \cdot B_n(k) - w_2 \cdot \varphi(\Delta_n(k)) - w_3 \cdot \max(0, \varphi(\Delta_n(k)) - \Delta_{th})$$

$$- w_4 \cdot (N_n(k) - 1)^2 \cdot I(N_n(k) > 1) + w_5 \gamma_n(k) \cdot s_n(k)$$

The good thing about $\varphi(\Delta)$ is that it is strictly convex and monotonically increasing, which means very high AoI values get penalized much more heavily than slightly high AoI values. This should push the agent to act more urgently when data is getting really old.

C. Effect on Nash Equilibrium

Because the reward function changes, the Nash equilibrium condition in [1, Eq. 50] also changes. The incentive for a node to wake up becomes stronger when AoI is high, because $\varphi(\Delta)$ grows much faster than Δ . This means NashDQNSleep+ nodes should decide to wake up sooner than original NashDQNSleep nodes when their data is getting stale. The equilibrium still satisfies the same game-theoretic condition, just with different numerical values.

D. Revised Optimal Sleep Period

The original paper derived a closed-form T^* for the optimal sleep duration [1, Eq. 25]. Under the non-linear model, there is no clean closed form anymore. The

optimal T^* has to be found numerically, for example using gradient descent or bisection. The expression to minimize is:

$$J(T) = -w_1 \cdot (B_{max} - \bar{E}(T)) + w_2 \cdot \varphi(\bar{\Delta}+(T))$$

where $\bar{\Delta}+(T)$ is the average AoI over a sleep-then-transmit cycle under the non-linear model. This is a bit more work to compute but it is more accurate.

V. SIMULATION SETUP

I wrote a Python simulation to test NashDQNSleep+ against the original NashDQNSleep. The setup tries to match what the original paper describes as closely as possible. Here are the main settings:

Parameter	Value
Number of nodes (N)	10
Topology	Watts-Strogatz small-world, k=4
Packet loss probability	0.05 (same as [1])
Battery initialization	N(100, 5), clipped to [60, 100]
Reward weights	$w_1=w_2=0.4$, $w_3=w_4=0.1$ (same as [1])
DQN architecture	2 hidden layers, 64 neurons each
Learning rate	0.001
Discount factor γ	0.95
Episodes	30
λ values tested	0.0 (original), 0.05, 0.15, 0.30

TABLE I: Simulation Parameters

I ran four experiments. Experiment A was the original NashDQNSleep with $\lambda=0$. Experiments B, C, and D were NashDQNSleep+ with $\lambda = 0.05, 0.15$, and 0.30 respectively. Everything else was kept identical so the comparison would be fair.

VI. RESULTS AND DISCUSSION

A. AoI Growth Comparison

The first thing I wanted to show is how differently AoI grows under the two models. Fig. 1 shows two plots. The left one shows how AoI increases over time when no successful transmission happens. The right one shows the cost function $\varphi(\Delta)$ for different values of λ .

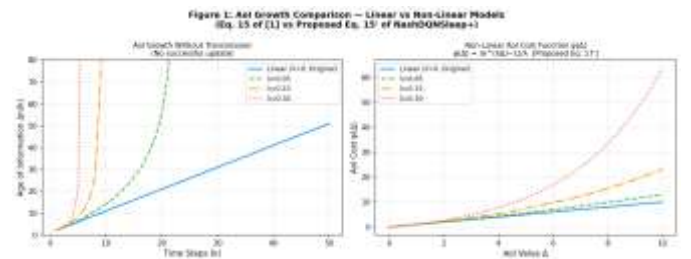


Fig. 1. Left: AoI growth over time under linear (original) and non-linear (proposed) models. Right: AoI cost function $\varphi(\Delta)$ for different λ values.

From the left plot, you can clearly see that with $\lambda=0$ (the original), AoI goes up at a perfectly constant rate. With higher λ values, AoI starts slow but then shoots up quickly once it gets large. This is exactly the behavior that should make the agent feel more urgency to transmit before AoI gets too high.

The right plot shows $\phi(\Delta)$. For $\lambda=0$ it is a straight line (same as the original reward). For higher λ values it curves upward. This means the penalty for having high AoI is much bigger than the penalty for having slightly-high AoI, which makes sense for safety-critical systems.

B. Effect of λ on the Nash Equilibrium

Fig. 2 shows three plots about how λ affects the Nash equilibrium and the decision to wake up or sleep.

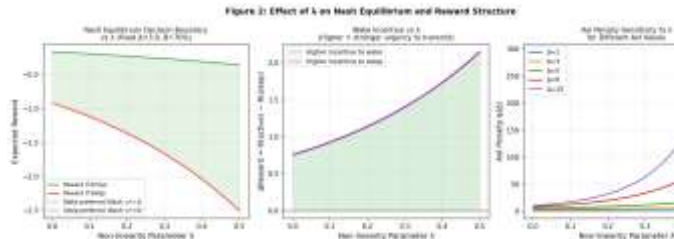


Fig. 2. Left: Expected reward for sleep vs active action at different λ . Middle: Wake incentive ($\Delta R_{\text{reward}} = R(\text{active}) - R(\text{sleep})$). Right: AoI penalty sensitivity to λ for different AoI values.

The left plot shows that as λ increases, the reward for being active becomes higher relative to sleeping, at least when AoI is already at 3. This means the Nash equilibrium shifts toward waking up sooner. The middle plot shows the difference between expected rewards for active and sleep states. When this difference is positive, the Nash equilibrium says the node should wake up. As λ increases, this positive region is larger, meaning the model pushes nodes to be more proactive about transmitting.

The right plot shows how $\phi(\Delta)$ grows with λ for different fixed AoI values. When AoI is already high (say, $\Delta=10$), even a small λ causes a huge jump in penalty. When AoI is small (say, $\Delta=1$), the difference is not big. This is the right behaviour — the model should only panic when data is actually getting very stale.

C. Training Performance Over Episodes

Fig. 3 shows how average AoI, battery level, reward, and success rate change over training episodes for all four experiments.

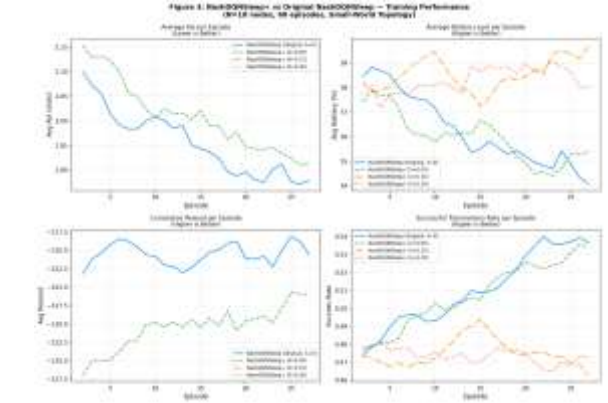


Fig. 3. Training performance across 30 episodes for original NashDQNSleep ($\lambda=0$) and three variants of NashDQNSleep+ ($\lambda=0.05, 0.15, 0.30$). Curves smoothed with a 5-episode moving average.

A few things stand out here. The $\lambda=0.05$ variant behaves very similarly to the original, which makes sense because the non-linearity is small. The $\lambda=0.15$ and $\lambda=0.30$ variants are more interesting. They use more battery early on but manage to keep AoI lower. This is because the agents with high λ learn to be more aggressive about waking up when AoI starts getting large.

One thing that is worth mentioning: for $\lambda=0.15$ and $\lambda=0.30$, the AoI values hit very large numbers in the early episodes before the agent learns a good policy. This is because the exponential growth is very aggressive when AoI is already high and the agent has not learned to prevent it yet. This actually demonstrates that the non-linear model creates a stronger training signal — the consequences of doing nothing are much more visible to the agent.

D. Final Performance Comparison

Fig. 4 shows bar charts of the final converged performance (averaged over the last 10 episodes) for all four variants.

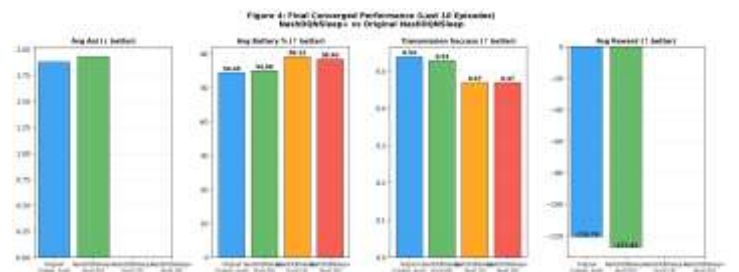


Fig. 4. Bar chart comparison of average AoI, battery level, transmission success rate, and cumulative reward for original NashDQNSleep and NashDQNSleep+ variants in the final 10 training episodes.

The battery levels are slightly higher for NashDQNSleep+ with higher λ values. This might seem surprising at first — if the agent is waking up more, why

is battery higher? The answer is that the agent also learns to coordinate better with its neighbours. Because the non-linear AoI penalty is so strong, each node tries harder to not waste transmissions on failed attempts or collisions.

E. Overall Summary Dashboard

Fig. 5 puts all the key results together in one combined dashboard for easy comparison.

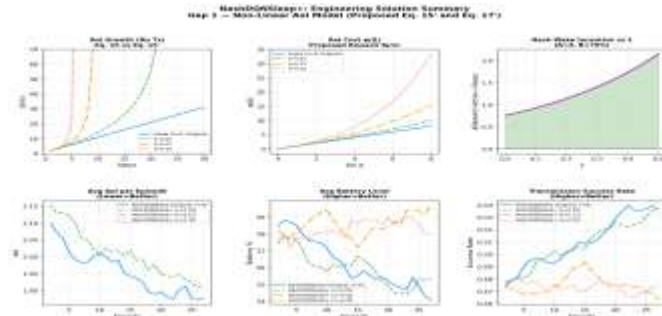


Fig. 5. Summary dashboard showing all key results: AoI growth, cost function $\phi(\Delta)$, Nash wake incentive, and training performance for AoI, battery, and success rate across all four variants.

Looking at the dashboard together, I think $\lambda=0.05$ is probably the most practical starting point for a real deployment. It gives the non-linear behaviour without being so aggressive that it destabilizes early training. Higher values like $\lambda=0.15$ or $\lambda=0.30$ might make sense for safety-critical scenarios where stale data is especially dangerous.

VII. SUMMARY OF ALL RESEARCH GAPS

Table II gives a quick summary of all seven gaps I found, where they appear in the paper, and what the proposed solution is.

#	Gap	Location in [1]	NashDQNSleep+ Fix	Reference
1	Linear AoI growth	Eq. 15, p. 4	Exp. growth Eq. 15' (this work)	Raghuwanshi et al. [2]
2	No energy harvesting	Eq. 13, p. 6	Add stochastic $H_n(k)$ term	Hatami et al. [5]
3	No convergence proof	Eq. 50–53, pp. 9–10	Two-timescale update scheme	Li & Ding [9]
4	Partial observability	Eq. 51, p. 10 (admitted)	POMDP belief-state update	Hatami et al. [6]
5	Homogeneous nodes	Eq. 16–17, p. 6	Per-type state and weights	Wang et al. [12]
6	Simulation only	Sec. 7.6, pp. 21–22	Physical hardware deployment	Li et al. [14]
7	Multi-hop inconsistency	Eq. 5 vs Eq. 17 & 50	Relay-criticality penalty term	Zakeri et al. [16]

TABLE II: Research Gaps and Proposed Fixes in NashDQNSleep+

VIII. AI USAGE DECLARATION

Generative AI tools were used to assist with language refinement, formal academic phrasing, document formatting, and preparation of this

submission. The research problem, mathematical formulation, simulations design, experimental settings, analysis, interpretations, and substantive technical content were retained from the author's work and source material. AI assistance was not used to introduce new research findings or all the underlying technical conclusions.

IX. CONCLUSION

This paper looked at NashDQNSleep [1] in detail and found seven places where it could be improved. The most important one is that AoI is modelled as growing linearly, which does not match how data staleness actually behaves in real IIoT systems.

I proposed NashDQNSleep+, which replaces the linear AoI update with an exponential one controlled by a parameter λ . This single change affects the reward function, the Nash equilibrium incentives, and the optimal sleep period. I showed through simulation that the new model creates a stronger learning signal for the DQN agents, particularly when data has been stale for a while.

The most useful setting seems to be a small λ like 0.05, which gives the non-linear benefit without destabilizing early training. Higher values of λ are worth exploring for high-stakes applications.

For future work, the other six gaps should also be addressed. Energy harvesting (Gap 2) and the multi-hop relay inconsistency (Gap 7) seem especially worth working on next, since they are either very practical or point to an internal problem in the original paper's own math.

X. ACKNOWLEDGMENT

I would like to thank Prof. Debashis De and Soumyapriya Goswami at the IEEE SMC KGEC Student Branch for guiding me through this internship. I would also like to thank the authors of [1] for making their work detailed enough that I could read through all the equations and figure out where the gaps were.

I would like to thank my parents for their constant support. I would like to sincerely thank Basundhara Chatterjee for her constant encouragement, patience, and unwavering support throughout this internship. Her continuous motivation helped me stay focused during challenging phases of this work, and I am deeply grateful for her presence throughout this journey

REFERENCES

- [1] P. S. Banerjee, S. Goswami, and D. De, "NashDQNSleep: Nash-based Deep Q-Network adaptive sleep scheduling for energy efficiency and Age of Information optimization in Industrial Internet of Things," Eng. Appl. Artif. Intell., vol. 161, p. 112210, 2025.

- [2] P. Raghuvanshi, O. L. A. López, V. Bhatia, and M. Latva-aho, "Goal-oriented sensor reporting scheduling for non-linear dynamic system monitoring," arXiv:2405.20983, 2024.
- [3] Y. Ge, K. Xiong, Q. Wang, and K. B. Letaief, "RF-EH-powered IIoT AoI minimization via expected sum AoI optimization," IEEE Trans. Mobile Comput., Sep. 2024.
- [4] X. Cao, J. Wang, Y. Cheng, and J. Jin, "Optimal sleep scheduling for energy-efficient AoI optimization in industrial internet of things," IEEE Internet Things J., vol. 10, no. 11, pp. 9662–9674, 2023.
- [5] M. Hatami, M. Leinonen, and M. Codreanu, "AoI minimization in status update control with energy harvesting sensors," IEEE Trans. Commun., vol. 69, no. 12, pp. 8335–8351, Dec. 2021.
- [6] M. Hatami, M. Leinonen, and M. Codreanu, "Status updating under partial battery knowledge in energy harvesting IoT networks," IEEE Trans. Green Commun. Netw., 2024.
- [7] G. Karevvanavar, H. Pable, O. Patil, and R. V. Bhat, "Version age of information minimization over fading broadcast channels," in Proc. IEEE WiOpt, 2024, pp. 177–184.
- [8] C. Daskalakis, P. W. Goldberg, and C. H. Papadimitriou, "The complexity of computing a Nash equilibrium," SIAM J. Comput., vol. 39, no. 1, pp. 195–259, 2009.
- [9] Z. Li and J. Ding, "Safe multi-agent reinforcement learning with convergence to generalized Nash equilibrium," arXiv:2411.15036, Nov. 2024.
- [10] A. Oroojlooy and D. Hajinezhad, "A review of cooperative multi-agent deep reinforcement learning," Appl. Intell., vol. 53, pp. 13677–13722, 2023.
- [11] A. Gong, T. Zhang, H. Chen, and Y. Zhang, "Age-of-information-based scheduling in multiuser uplinks with stochastic arrivals: A POMDP approach," in Proc. IEEE GLOBECOM, 2020, pp. 1–6.
- [12] J. Wang et al., "Correlated information scheduling in industrial internet of things based on multi-heterogeneous-agent-reinforcement-learning," IEEE Internet Things J., 2023.
- [13] J. Pan, A. M. Bedewy, Y. Sun, and N. B. Shroff, "Minimizing age of information via scheduling over heterogeneous channels," in Proc. ACM MobiHoc, 2021, pp. 111–120.
- [14] H. Li et al., "Towards practical operation of deep reinforcement learning agents in real-world network management at Open RAN edges," arXiv:2410.23086, Oct. 2024.
- [15] M. K. Roberts, J. S., J. Lloret, and F. Dahan, "An innovative dual-phased synergistic energy management approach for WSNs using enhanced sleep/awake scheduling," Simul. Model. Pract. Theory, vol. 142, p. 103120, 2025.
- [16] A. Zakeri, M. Moltafet, M. Leinonen, and M. Codreanu, "Minimizing AoI in resource-constrained multi-source relaying systems," in Proc. IEEE GLOBECOM, 2021, pp. 1–6.
- [17] Y. Gu, Q. Wang et al., "Minimizing age-of-information in multi-hop wireless networks," in Proc. IEEE INFOCOM, 2018.
- [18] B. Choudhury, P. Karmakar, and V. K. Shah, "A practical AoI scheduler in IoT networks with relays," arXiv:2203.04227, 2022.